

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>Н2</u>	<u>Программная инженерия и интеллектуальные системы</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Проектирование архитектуры программных средств</u>	
	наименование дисциплины	

УЧЕБНО–ПРАКТИЧЕСКАЯ РАБОТА

3

номер задания (при наличии)

FLOWCHART-ДИГРАММА

при наличии указать тему учебно–практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О729Б

Барышников О.А.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

Зими́на Д.В.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

г. Санкт–Петербург

20 26 г.

СОДЕРЖАНИЕ

1 Flowchart-диаграмма	3
2 Описание flowchart-диаграммы	5

1 Flowchart-диаграмма

На рисунках 1-3 изображена Cross-functional flowchart диаграмма.

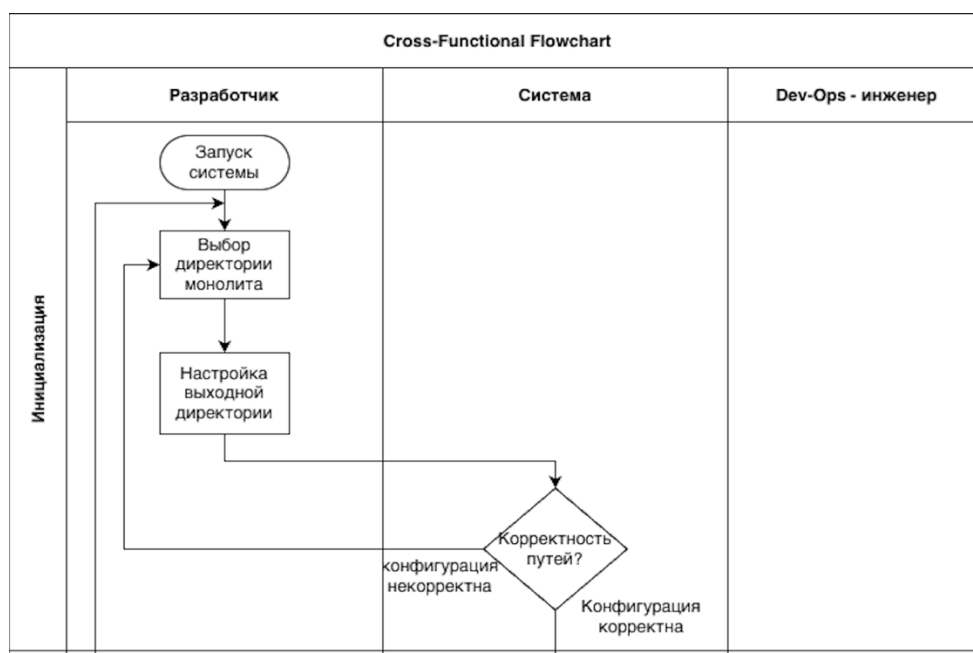


Рисунок 1 – Flowchart-диаграмма

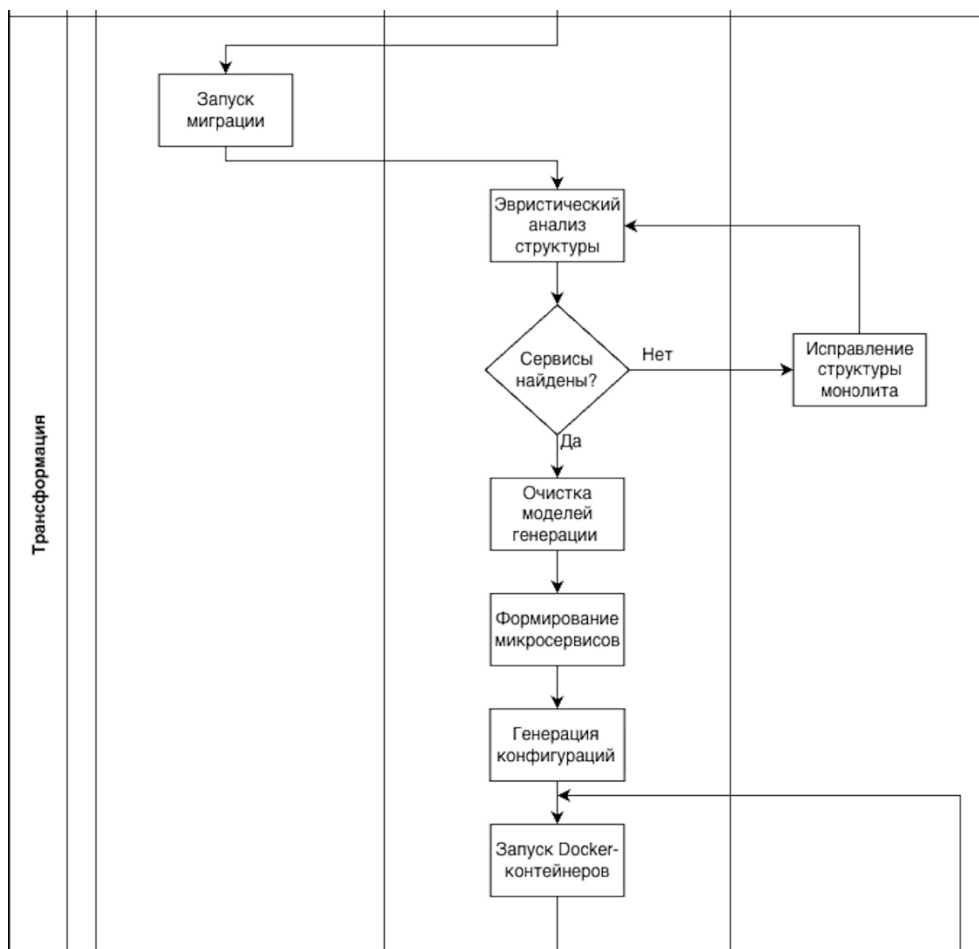


Рисунок 2 – Flowchart-диаграмма

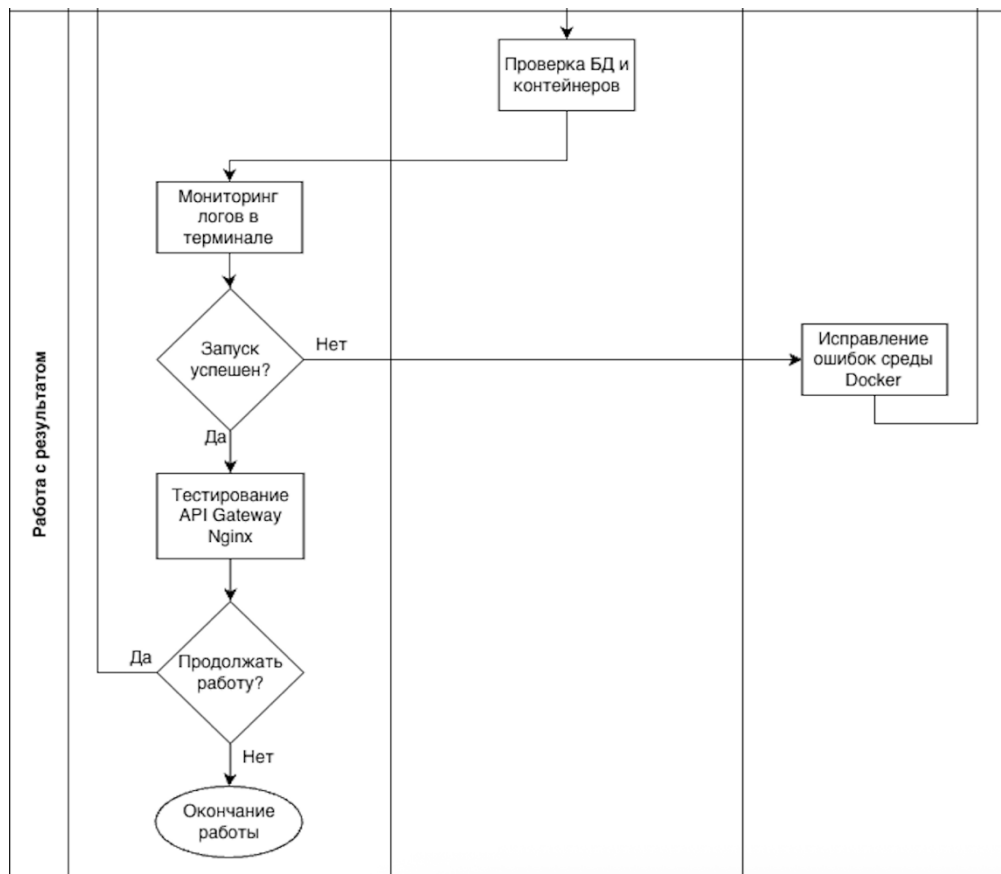


Рисунок 3 – Flowchart-диаграмма

2 Описание flowchart-диаграммы

Процесс работы программного инструмента автоматизированной декомпозиции монолита (АММ-Docker) разделен на три ключевых этапа: инициализация, трансформация (обработка) и работа с результатом. Процесс начинается с этапа инициализации. На этом этапе разработчик осуществляет запуск графического интерфейса системы. Затем он задает входные параметры: выбирает директорию исходного кода монолитного приложения и настраивает путь к выходной директории для сохранения сгенерированных микросервисов. После получения параметров система автоматически проверяет корректность путей и наличие исходного кода Python. Если конфигурация некорректна (например, указана пустая папка), процесс возвращается на этап настройки для исправления путей; при корректной конфигурации система переходит к этапу обработки.

Этап трансформации (обработки) представляет собой основной конвейер работы ядра АММ-Docker. Процесс инициируется разработчиком путем нажатия кнопки запуска миграции. Сначала система выполняет эвристический анализ файловой структуры монолита. Система проверяет наличие потенциальных сервисов. Если независимые модули не найдены, процесс прерывается, и DevOps-инженер выполняет ручное исправление структуры или зависимостей монолита.

При успешном нахождении сервисов система последовательно выполняет:

- семантическую очистку моделей баз данных и генерацию прокси-заглушек (Stub);
- физическое разделение кода и формирование файлов микросервисов;
- генерацию инфраструктурных конфигураций (Dockerfile, nginx.conf, docker-compose.yaml).

Завершается этап автоматической отправкой команд в подсистему Docker для сборки и запуска изолированных контейнеров.

На этапе работы с результатом система осуществляет проверку работоспособности развернутой инфраструктуры (например, healthcheck базы данных PostgreSQL). Параллельно разработчик осуществляет мониторинг логов через встроенный эмулятор терминала. Если в процессе сборки контейнеров или

старта сервисов возникают системные ошибки (например, конфликт портов или отсутствие Docker), DevOps-инженер выполняет исправление ошибок среды выполнения.

При успешном запуске всех узлов разработчик проводит тестирование маршрутизации HTTP-запросов через единую точку входа — API Gateway (Nginx). После анализа работоспособности полученной микросервисной сети принимается решение о необходимости продолжения работы. Если требуется обработка другого проекта, процесс возвращается к этапу инициализации. В противном случае работа с программой завершается.

Вся последовательность действий графически оформлена с использованием стандартной нотации: логические проверки и ветвления представлены условными блоками (ромбами), действия — прямоугольными блоками, начало и конец процесса обозначены овалами, а направленные стрелки отражают причинно-следственную связь и переходы управления между участниками (дорожками) процесса.